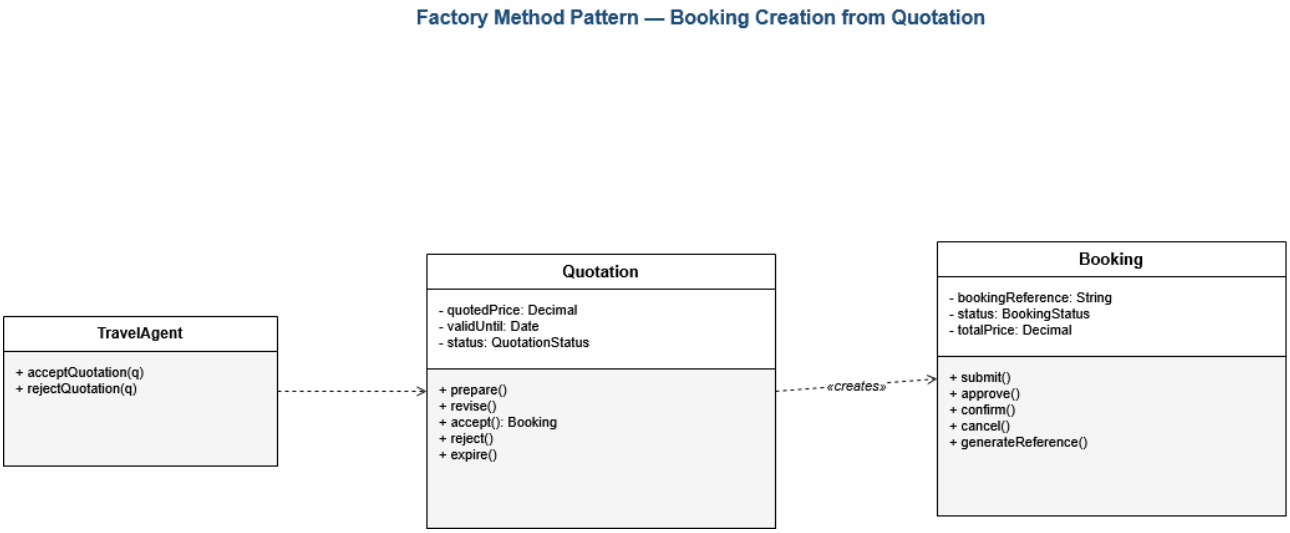
Design Patterns — Justification & Class Diagrams

1. Factory Method

Creational

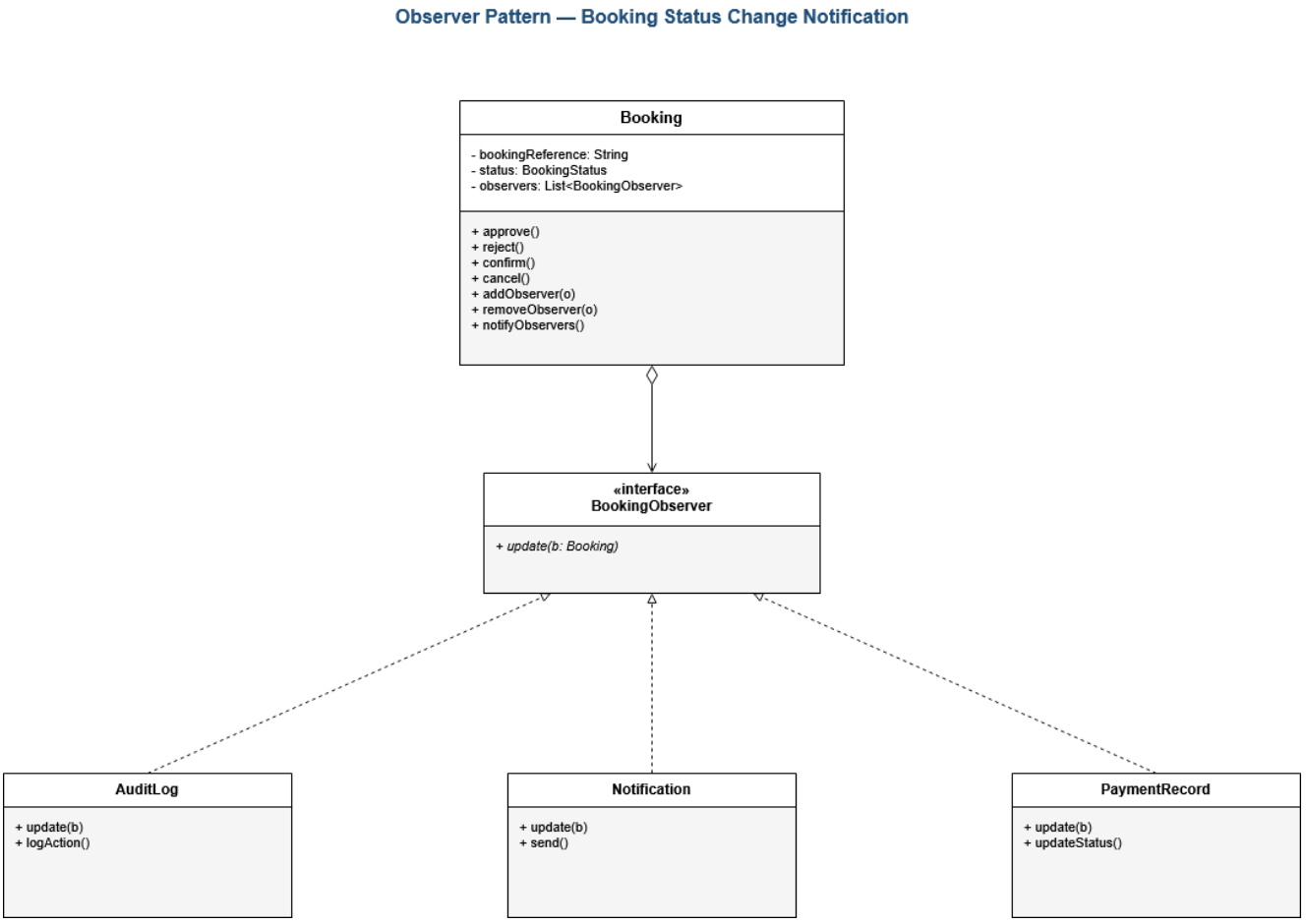
The Factory Method pattern is already explicitly present in our class specification through the Quotation.accept(): Booking method. A Booking in our system is a complex entity with multiple attributes that must be initialized consistently from the originating Quotation, including the quoted price, traveler details, package reference, and the unique booking reference. Rather than allowing Booking objects to be constructed arbitrarily from anywhere in the system, we channel all creation through this factory method on Quotation. This ensures every Booking is properly initialized, validated against the originating quotation's terms, and traceable back to its source. The pattern fits naturally because our travel agency operates with a formal quotation workflow: customers receive a written estimate before committing, and a Booking can only emerge from an accepted Quotation. The Factory Method enforces this business rule structurally rather than relying on developer discipline.



2. Observer

Behavioral

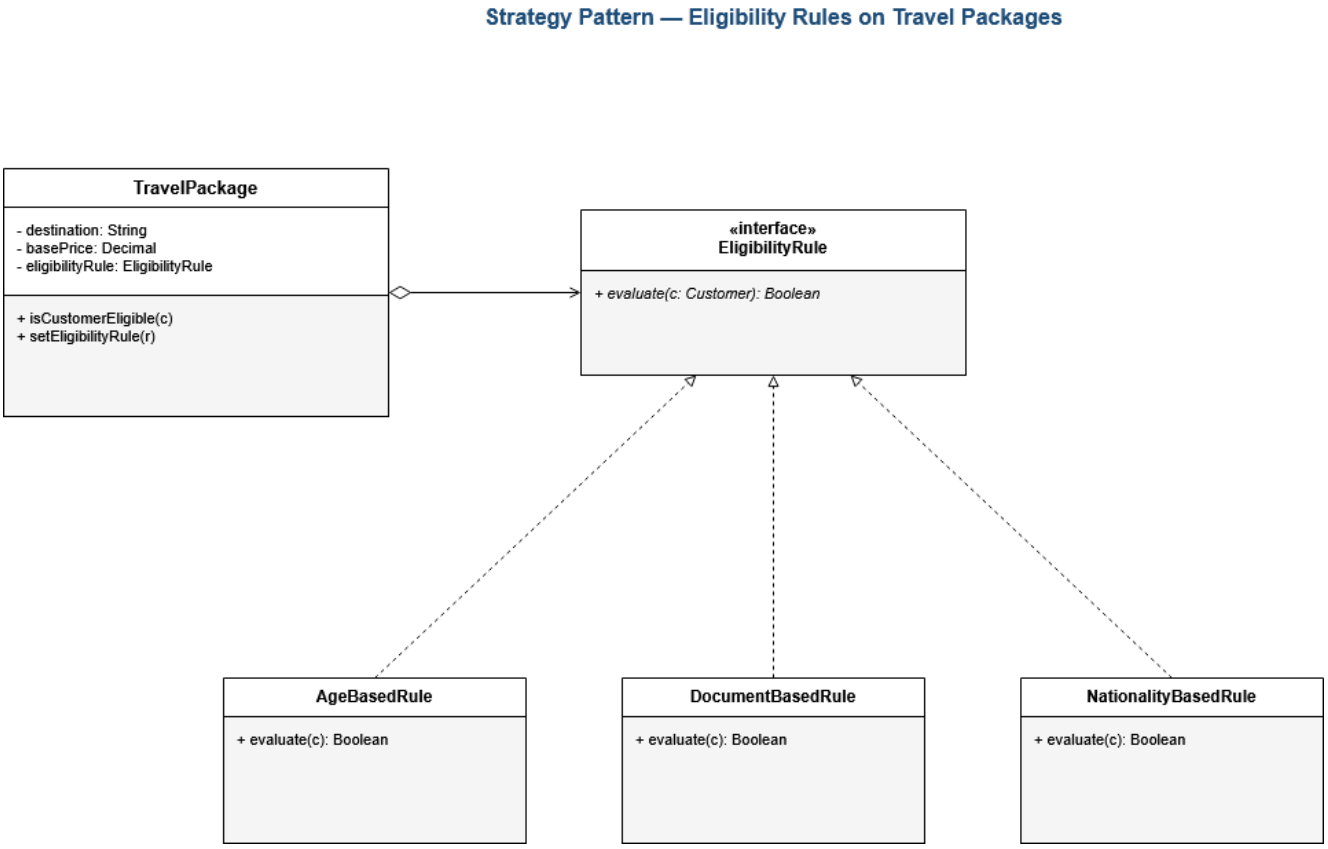
The Observer pattern formalizes a behavior already implicit in our class structure: whenever a Booking changes status, several other classes must react in a coordinated way. The AuditLog records the action for compliance and traceability, the Notification class generates and sends communications to the customer, and the PaymentRecord updates its status to trigger receipts or refund flagging. Without the pattern, the Booking class would need direct references to all three of these classes and call them explicitly on every status change, creating tight coupling and fragile code. By introducing a BookingObserver interface that AuditLog, Notification, and PaymentRecord implement, the Booking simply notifies its registered observers when its state changes. Each observer handles its own responsibility independently. This decoupling makes it trivial to add new reactions to booking events later, such as waitlist notifications, without modifying the Booking class itself.



3. Strategy

Behavioral

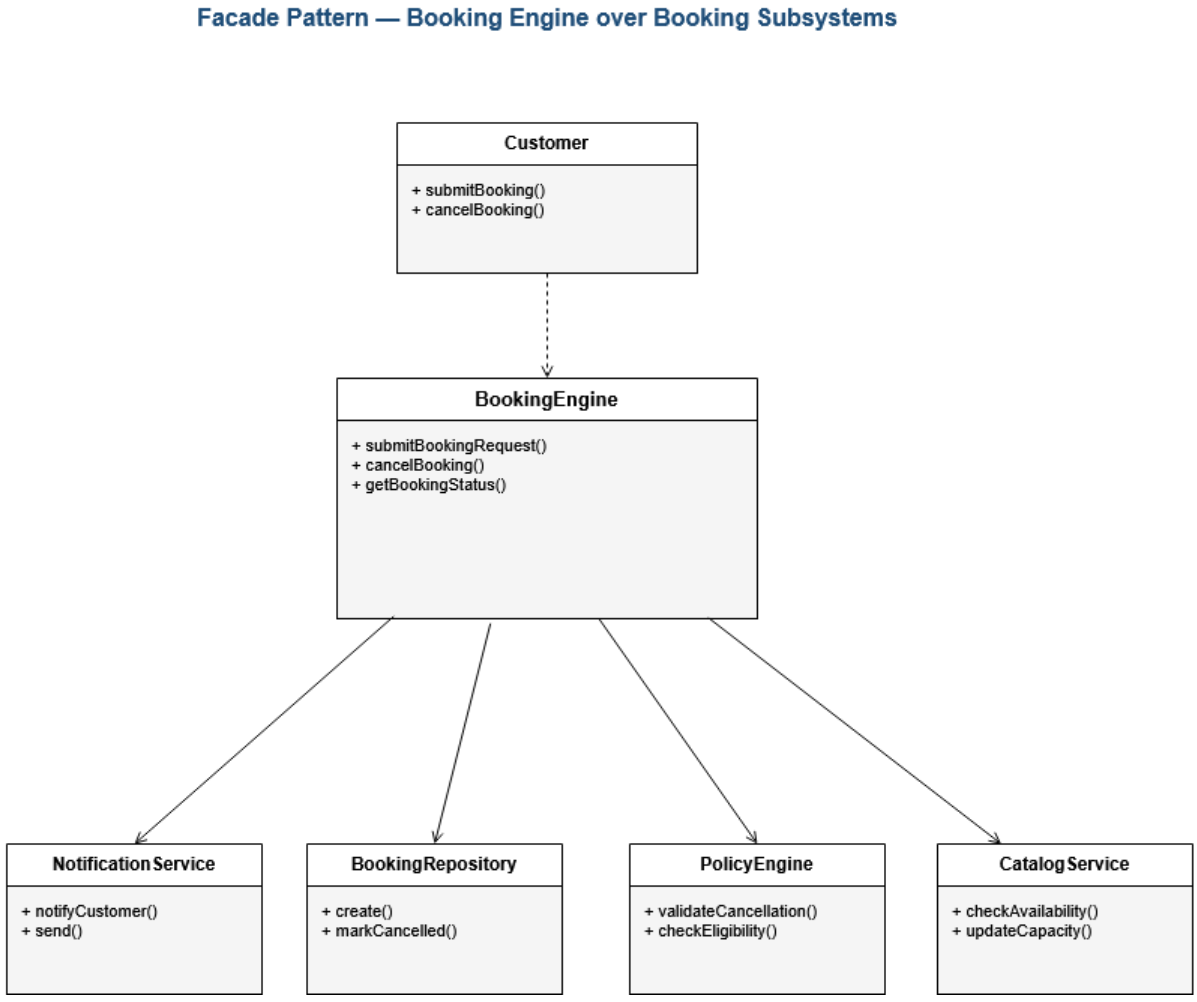
The Strategy pattern fits our TravelPackage class because different packages require different eligibility validation logic. A family-oriented package checks for accompanying adults, a senior tour validates age ranges, an international package verifies passport and visa documents, and some standard packages have no restrictions at all. Our class specification already includes an EligibilityRule class with an EligibilityRuleType enum, which hints at multiple rule variants. The Strategy pattern formalizes this by defining an EligibilityRule interface with concrete implementations such as AgeBasedRule, DocumentBasedRule, and NationalityBasedRule. The TravelPackage holds a reference to whichever rule applies and delegates eligibility checks to it. Adding a new rule type, for example a VaccinationBasedRule for destinations with health requirements, requires no changes to the TravelPackage class. The eligibility logic stays isolated and pluggable.



4. Facade

Structural

The Facade pattern is reflected in our BookingEngine, which is one of the named system components we identified throughout the project. Submitting a booking is a multi-step operation that touches several subsystems: the CatalogService for package availability and capacity, the PolicyEngine for eligibility validation, the BookingRepository for persistence and queue routing, and the NotificationService for customer communication. Exposing all four of these subsystems directly to the Customer or the Travel Agent would force them to coordinate operations across multiple classes for every booking action. Instead, the BookingEngine acts as a Facade: it provides simple, business-meaningful methods such as submitBookingRequest() and cancelBooking() that internally coordinate the subsystem calls in the correct sequence. The client interacts only with the Facade and never needs to know about the underlying complexity. This simplifies the client code, reduces coupling, and makes the booking workflow easier to maintain as individual subsystems evolve.



5. Template Method

Behavioral

The Template Method pattern fits our Employee class hierarchy naturally. Every employee follows the same daily shift structure: log in, load their dashboard, perform their role-specific tasks, log activity, and log out. The first, fourth, and fifth steps are identical for every employee regardless of role. The second and third steps, however, are completely different depending on whether the employee is a Travel Agent reviewing bookings, Finance Staff processing payments, HR Staff managing leave requests, Operations Staff coordinating schedules, or a Manager reviewing financial reports. The abstract Employee class defines the template method performDailyShift() that orchestrates the sequence, with login() logout() and logActivity() as concrete shared methods, while loadDashboard() and performRoleTasks() are abstract and overridden by each concrete subclass. This keeps the workflow consistent across all roles, prevents duplicate code in subclasses, and makes it straightforward to onboard a new employee type later by simply implementing the two abstract methods.